Ideation Phase

Define the Problem Statements

Date	28-10-2026
Team ID	SWTID-2026-7875
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value

1. Primary Problem Statements

Problem Statement 1: Data Leakage Across Academic Branches / Departments

- **Description:** In a multi-department system, users from one branch (e.g., ECE or CSE) can view records from other branches (e.g., EEE) if access controls only evaluate table-level access rather than record-level field values.
- **Impact:** Confidentiality breach and failure to satisfy departmental data segregation mandates.
- **Lab Context:** The custom table u_institution_details holds records for ECE, EEE, and CSE branches. Without script-controlled ACLs, all users with list access see all branch records.

Problem Statement 2: UI-Only Controls vs. Backend API Vulnerability

- **Description:** Relying on UI Policies, Client Scripts, or Module Filters to restrict visibility only conceals data on standard forms/lists, leaving backend database endpoints accessible via Direct URL navigation (.list), Web Service APIs, or background scripts.
- **Impact:** False sense of security; savvy users can bypass UI restrictions to view or modify database rows.
- **Lab Context:** Users can access list views directly via URL parameter manipulation (u_institution_details.list). Access must be enforced at the database security layer using server-side Access Control Lists (ACLs).

Problem Statement 3: Lack of Granular CRUD Operation Controls

- **Description:** Access permissions are frequently granted as blanket rights.

Users who need Read access are often granted implicit Write, Create, or Delete capabilities, or vice versa.

- **Impact:** Unintended record deletion, unauthorized record creation, or data corruption by non-administrative personnel.
- **Lab Context:** Operational security requires explicit separation:
  - **Read Access:** Restricted to branch-specific users (e.g., role bb1 + Branch is EEE).
  - **Create Access:** Governed by creation role bb2.
  - **Write Access:** Governed by modification role bb3.
  - **Delete Access:** Governed by deletion role bb4.

Problem Statement 4: Static Access Rules vs. Context-Aware Evaluation

- **Description:** Standard ACLs evaluate static conditions (e.g., user has role X), but cannot evaluate dynamic runtime conditions like field values, session parameters, or contextual business logic.
- **Impact:** Inflexible security models that fail to support complex business criteria (e.g., "Allow access ONLY if user branch equals record branch").
- **Lab Context:** Scripted ACL conditions (gs.hasRole() combined with field criteria Branch == EEE) evaluate both static roles and dynamic field values together before returning an access decision (true/false).

2. Problem & Solution Mapping Matrix

Focus Area	Identified Gap / Problem Statement	Desired Solution State
Record Segregation	Cross-departmental data viewing	Scripted Read ACL enforcing Branch == EEE for branch users while allowing global access for Administrators.
Operation Separation	Over-privileged user actions	Independent ACL definitions for create, write, and delete operations mapped to dedicated roles (bb2, bb3, bb4).
Platform Oversight	Risk of lockouts or admin restrictions	Elevated privilege handling (security_admin) and scripted admin overrides.

3. Next Steps

1. Refactor granular roles (bb1–bb4) into a hierarchical role structure.
2. Upgrade static role checks to dynamic user profile lookups (sys_user).
3. Implement Before Query Business Rules to prevent ACL constraint error messages on list views.